

MARAN PMS — Hardening y Fixes de Calidad

PROMPT: Seguridad, autenticación, errores silenciosos e IVA restaurant

FIX 1 — `requireAuth` en todos los endpoints de la API

Actualmente solo los endpoints del módulo contable tienen `requireAuth`. Todos los demás (reservas, habitaciones, restaurant, spa, eventos, etc.) aceptan requests sin estar autenticado.

En `server/routes.ts`, agregar `requireAuth` como segundo argumento en todos los endpoints que no lo tengan:

```
// Patrón actual (sin auth):
app.get("/api/reservations", async (req, res) => { ... });

// Correcto:
app.get("/api/reservations", requireAuth, async (req, res) => { ... });
```

Lista de módulos a corregir — aplicar `requireAuth` en todos sus endpoints:

- **Reservas:** `/api/reservations`, `/api/planning`, `/api/reservations/:id/*`
- **Habitaciones:** `/api/rooms`, `/api/room-types`, `/api/bed-types`, `/api/rate-plans`, `/api/packages`
- **Huéspedes:** `/api/guests`, `/api/companies`, `/api/agencies`
- **Restaurant:** `/api/restaurant/*`
- **Spa:** `/api/spa/*`
- **Eventos:** `/api/events/*`
- **Housekeeping:** `/api/housekeeping/*`
- **Mantenimiento:** `/api/maintenance/*`

- **Inventario:** `/api/inventory/*`
- **Notificaciones:** `/api/notifications/*`
- **Dashboard:** `/api/dashboard/*` , `/api/account-summary`
- **Admin:** `/api/admin/*`
- **Grupos:** `/api/groups/*`
- **OTAs:** `/api/ota-channels/*` , `/api/ota-reservations/*`
- **Reseñas:** `/api/reviews/*`
- **Hospitalidad:** `/api/hospitality/*`
- **Web check-in (admin):** `/api/web-checkin/list` , `/api/web-checkin/generate/:reservationId` , `/api/web-checkin/:reservationId/status`

Excepciones — estos SÍ deben quedar públicos:

GET <code>/api/public/web-checkin/:token</code>	← web checkin del huésped
POST <code>/api/public/web-checkin/:token</code>	← web checkin del huésped
POST <code>/api/auth/login</code>	← login
POST <code>/api/auth/logout</code>	← logout
GET <code>/api/auth/session</code>	← verificar sesión
POST <code>/api/webhook/chatbot</code>	← webhook MARA (usa su propio secret)

FIX 2 — Blindar `/api/auth/setup`

El endpoint `POST /api/auth/setup` es público y crea un usuario admin. Tiene protección solo si ya hay usuarios con password, pero el password `"maran2026"` está hardcodeado como fallback.

Cambio:

```
// Antes:
app.post("/api/auth/setup", async (req, res) => {
  // ... crea admin con password hardcodeado
});

// Después — deshabilitar completamente en producción:
app.post("/api/auth/setup", async (req, res) => {
```

```
// Solo permitir en desarrollo
if (process.env.NODE_ENV === "production") {
  return res.status(404).json({ message: "Not found" });
}

const allUsers = await db.select().from(systemUsers);
const anyUserWithPassword = allUsers.some(u => u.password !== null);
if (anyUserWithPassword) {
  return res.status(400).json({ message: "Setup ya completado." });
}
// ... resto del setup
});
```

FIX 3 — SESSION_SECRET obligatorio sin fallback hardcodedo

```
// Antes:
secret: process.env.SESSION_SECRET || "maran-suite-secret-key",

// Después:
if (!process.env.SESSION_SECRET) {
  if (process.env.NODE_ENV === "production") {
    throw new Error("SESSION_SECRET es obligatorio en producción");
  }
  console.warn("⚠️ SESSION_SECRET no definido — usando clave de desarrollo. NO u
}

secret: process.env.SESSION_SECRET || "dev-only-maran-secret-do-not-use-in-prod",
```

FIX 4 — Ruta /settings: corregir o redirigir

La ruta `/settings` aparece en el menú lateral (`href: "/settings"`) pero no existe como `<Route>` en el router. Provoca un 404.

Opción A — Si hay una página de configuración del sistema, registrar la ruta:

```
// En el Router, agregar:
<Route path="/settings" component={SettingsPage} />
```

Opción B — Si la configuración ya está en `/administration`, redirigir:

```
<Route path="/settings">
  { () => { window.location.replace("/administration"); return null; }}
</Route>
```

Opción C — Cambiar el href en el menú lateral directamente a `/administration`:

```
// En la config del sidebar:
// Antes:
{ label: "Configuración", icon: Settings, href: "/settings" }

// Después:
{ label: "Configuración", icon: Settings, href: "/administration" }
```

Aplicar la Opción C — es la más simple y directa ya que `/administration` ya existe y tiene la configuración del sistema.

FIX 5 — `/source-code`: restringir a admin

La página `/source-code` es accesible para cualquier usuario logueado. Debe ser solo para administradores.

En el componente `SourceCodePage`:

```
export default function SourceCodePage() {
  const { user } = useAuth();

  if (user?.role !== "admin") {
    return (
      <div className="flex items-center justify-center h-full">
        <p className="text-muted-foreground">Acceso restringido.</p>
      </div>
    );
  }

  // ... resto del componente
}
```

En el sidebar — ocultar el ítem para no-admins:

```
// En la definición del menú, agregar condición de role:
{ label: "Código fuente", icon: Code2, href: "/source-code", adminOnly: true }

// Y en el render del sidebar filtrar:
.filter(item => !item.adminOnly || user?.role === "admin")
```

FIX 6 — IVA en el restaurant: desglosar en lugar de agregar

Problema: Al agregar un ítem a una comanda, el backend calcula:

```
const tax = newSubtotal * 0.21;
total = newSubtotal + tax; // ← precio del menú + 21% encima = precio inflado
```

Esto significa que si un ítem tiene precio \$1.000 en el menú, el total de la mesa queda en \$1.210 . Incorrecto — el precio del menú ya tiene IVA incluido.

Corrección: El subtotal ya ES el precio con IVA. El `tax` es la porción de IVA dentro del precio, no algo que se agrega encima.

```
// Antes (incorrecto):
const newSubtotal = orderItems.reduce((sum, i) => sum + parseFloat(i.subtotal), 0)
const tax = newSubtotal * 0.21;
const total = newSubtotal + tax;

// Después (correcto — precio con IVA incluido):
const total = orderItems.reduce((sum, i) => sum + parseFloat(i.subtotal), 0);
const neto = parseFloat((total / 1.21).toFixed(2));
const tax = parseFloat((total - neto).toFixed(2));

await storage.updateRestaurantOrder(req.params.orderId, {
  subtotal: neto.toFixed(2), // neto sin IVA (para el desglose contable)
  tax: tax.toFixed(2),      // IVA contenido
  total: total.toFixed(2),  // precio final que paga el cliente
  status: "in_progress",
});
```

Este mismo patrón aplica en **todos los lugares** del backend donde se calcula `* 0.21` para el restaurant. Buscar y reemplazar todas las ocurrencias de `newSubtotal * 0.21` en `server/routes.ts`.

En el frontend, si se muestra el desglose del ticket (subtotal + IVA + total), asegurarse de que el total no cambie — solo cambia la forma en que se presenta el desglose.

FIX 7 — Catches vacíos en pagos y movimientos de caja

Hay 5 bloques `catch (e) {}` que silencian errores en operaciones de registro de movimientos de caja. Si `registerCashMovement()` falla, el sistema no lo sabe ni lo informa.

Posiciones afectadas:

- Registro de pago en reservación → `registerCashMovement("reception", ...)`
- Cierre de orden en restaurant → `registerCashMovement("restaurant", ...)`
- Pago de cuenta spa → `registerCashMovement("spa", ...)`
- Limpieza de datos en seed/reset

Reemplazar los `catch (e) {}` por:

```
// Antes:
} catch (e) {}

// Después:
} catch (e) {
  console.error("Error registrando movimiento de caja:", e);
  // No propagar — el pago principal ya fue registrado correctamente
  // pero loguear para poder debuguear si hay inconsistencias en caja
}
```

Los catches en la limpieza del seed pueden quedar vacíos — son operaciones de reset de datos y no son críticos.

FIX 8 — onError en mutations del frontend

95 mutations no tienen `onError`. Si la llamada al backend falla, el usuario no ve ningún mensaje de error.

Agregar un helper global de error:

En `client/src/lib/utils.ts` o en un archivo `client/src/lib/mutations.ts` :

```
export function defaultMutationError(error: any, fallback = "Ocurrió un error") {  
  return error?.message || fallback;  
}
```

Patrón a aplicar en todas las mutations que no tengan `onError` :

```
// Antes:  
const mutation = useMutation({  
  mutationFn: (data) => apiRequest("POST", "/api/...", data),  
  onSuccess: () => {  
    toast({ title: "Guardado" });  
  },  
});  
  
// Después:  
const mutation = useMutation({  
  mutationFn: (data) => apiRequest("POST", "/api/...", data),  
  onSuccess: () => {  
    toast({ title: "Guardado" });  
  },  
  onError: (error: any) => {  
    toast({  
      title: "Error",  
      description: error?.message || "No se pudo completar la operación",  
      variant: "destructive",  
    });  
  },  
});
```

Módulos prioritarios donde aplicar primero (los más críticos):

1. `AdminCajaPage` — movimientos de caja
2. `CheckOutPage` — check-out
3. `GroupDetailPage` — grupos
4. `RatePlansPage` — tarifas
5. `SpaClientsPage` — clientes spa
6. `GuestsPage` — huéspedes

7. `InventoryPage` — inventario

8. Todos los demás módulos a continuación

NOTAS DE IMPLEMENTACIÓN

1. El **Fix 1** (`requireAuth`) es el más extenso en cantidad de líneas pero mecánico — es agregar `, requireAuth` en cada `app.get/post/patch/delete`. Se puede hacer con una búsqueda y reemplazo guiada.
2. El **Fix 6** (IVA restaurant) es el más delicado en lógica. Verificar después que los totales en el ticket y el folio de habitación (cuando se carga a habitación) muestren el mismo valor que antes.
3. Los **Fix 2, 3, 4, 5** son cambios pequeños y de bajo riesgo.
4. El **Fix 8** (`onError`) puede aplicarse módulo por módulo sin romper nada — es solo agregar código.
5. **No incluido en este prompt — requiere diseño previo:** sistema de permisos granulares por acción (ver, crear, editar, cobrar, cerrar caja, emitir comprobante). Eso lo planificamos por separado antes de codificarlo.